Impact of Inference-Time Reasoning on LLM-Based Parallel Code Generation: a Case-Study of Kokkos

Venantius Sean Ardi Nugroho[1] and Shinobu Miwa[2]

[1] University of Electro-Communications
seannugroho8@gmail.com
[2] University of Electro-Communications
shinobu.miwa@uec.ac.jp

Abstract. Cost efficiency increases its importance in LLM-based code generation. This motivates us to analyze the impact of *inference-time reasoning* (i.e., effort control at inference), which has lately started to be supported by commercial LLMs, on LLM-generated Kokkos code. To compare the cost-efficiency of inference-time reasoning with multiple sampling, which has been widely used to improve LLM's answers, we use 4,800 Kokkos codes generated by the GPT-5.1 model with four (no/low/medium/high) effort levels. Our experimental results show that the use of a higher effort level increases the correctness of the generated code, but it also significantly increases the cost of code generation; therefore, in an environment with automated validation, using multiple sampling at the no effort level is the most beneficial in terms of cost efficiency. Furthermore, our experimental results show some cases where an increase in effort levels brings no performance improvement (often performance degradation) to the generated code. This suggests that LLM-based "efficient" parallel code generation is a fundamentally different challenge from LLM-based "correct" parallel code generation.

1 Introduction

Large language models (LLMs) are increasingly used in software development, and prior work reports measurable productivity gains in general programming tasks [12]. In contrast, generating *parallel* code remains substantially more difficult than generating sequential code, because parallel programming requires correct reasoning about task decomposition, data dependencies, synchronization, and mutual exclusion [9]. These challenges are amplified in low-resource parallel programming frameworks, where high-quality training examples are relatively scarce.

Kokkos is a representative low-resource setting for this problem. Compared with higher-resource frameworks such as OpenMP, LLM-generated Kokkos programs tend to show lower correctness and reliability, often due to syntax and parallel-logic failures [15]. Recent API controls for *inference-time reasoning* enable test-time compute to be adjusted (reasoning effort) without changing model

weights. This raises a practical question for LLM-assisted parallel code development: when generating parallel Kokkos code, is it better to allocate a budget to deeper reasoning per sample, or to generate more samples?

Motivated by this question, this paper examines the impact of inference-time reasoning on LLM-based parallel code generation through a focused Kokkos case study. We evaluate GPT-5.1 across multiple reasoning-effort settings and analyze their effects on correctness, failure modes, and parallelization behavior. We also compare reasoning-effort control with multiple sampling to characterize token-cost versus quality trade-offs under a fixed-budget perspective.

The main contributions of this study are as follows:

- We quantify how inference-time reasoning effort affects functional correctness (pass@k) and error distributions for GPT-5.1-generated Kokkos code.
- We examine whether increased reasoning effort improves parallelization quality, including observed CPU speedup behavior.
- We compare reasoning-effort control against multiple-sample generation to clarify practical trade-offs between token cost and solution quality.

The remainder of this paper is organized as follows. Section 2 summarizes related work. Sections 3 and 4 present the experimental methodology and results, respectively. Finally, Section 5 concludes the paper.

Results show that higher effort improves pass@1, but high effort is far more expensive, so accuracy gains are not always cost-proportional; under automated validation, sampling multiple no-effort Kokkos code is often more token-efficient. Error taxonomy is dominated by compilation failures, and several problems exhibit systematic 100% failure regardless of effort. For performance, passing codes demonstrate real parallelism (speedup rises with threads) but are constrained by strong-scaling limits, with anomalies such as a speedup drop at 32 threads (likely from simultaneous multithreading), low effort yielding the worst speedup, and occasional cases where no effort outperforms medium/high effort—reinforcing that functional correctness and parallel efficiency are distinct targets, and extra reasoning does not guarantee scalability.

2 Related Works

Within high-performance and parallel-programming contexts, previous studies consistently show a larger difficulty gap than in general sequential code generation. Godoy et al. evaluated GPT-based assistants for generating HPC kernels across programming models and reported mixed results in both correctness and portability, including challenges for Kokkos-like workflows [4, 5]. Nichols et al. introduced ParEval, a benchmark spanning OpenMP, MPI, and Kokkos, and demonstrated notable degradation in correctness for parallel tasks [9]. Their HPC-Coder study further suggests that domain-specific modeling and workflow design are often required for robust parallel-code generation [10]. Complementary OpenMP-focused user-side evaluations also report frequent practical failure modes in generated parallel code [8].

Table 1. System configuration

Name	Remarks
CPU	AMD Ryzen Threadripper PRO 3955WX (16C/32T, 3.9GHz)
Memory	64 GiB DDR4-3200

Recent work has increasingly examined low-resource settings and inference-time interventions, which allow LLMs to use extra computation during generation to think through a given complex problem before answering [11, 18]. For Kokkos, Singh et al. showed that test-time compute and prompting strategies can improve low-resource parallel-code generation [15]. Ellingwood et al. emphasized the need for both compile-time and run-time validation when assessing LLM-generated Kokkos programs [3]. Additionally, another study that evaluated LLM generated Kokkos code, such as the one conducted by Valero Lara et al, shows that through ChatHPC, a lightweight infrastructure/library that creates specialize LLM through fine tuning that they created, they were able to achieve high correctness on Kokkos parallelization and translation prompts [17]. For MPI, Tanaka et al. reported that GPT-4o still frequently fails in task decomposition [16]. Building on this literature, our study focuses specifically on inference-time reasoning control in GPT-5.1 and compares it directly with multiple sampling under a cost-aware Kokkos setting.

3 Experimental Methodology

3.1 Experimental System

Our experiments were performed on the compute node shown in Table 1. The node has one AMD Ryzen Threadripper PRO 3955WX CPU and 64 GiB DDR4-3200 memory. All Kokkos codes were executed with the OpenMP backend, enabling parallel execution on up to 16 physical cores. We used Kokkos 4.7.01. All LLM inference was performed via OpenAI's API with the GPT-5.1 model.

3.2 Benchmarks

We use the ParEval benchmark suite [9] to generate Kokkos code. ParEval consists of 12 problem types, with each having five representative problems, as shown in Table 2. Thus, ParEval has 60 problems in total.

We use the following four types of inference-time reasoning controls supported by OpenAI [11].

No effort: runs with no (or almost no) reasoning tokens allocated before answering.

Low effort: uses fewer reasoning tokens to prioritize speed and economical token usage.

Medium effort: the default setting for GPT-5.1; it keeps a balance between speed and reasoning accuracy.

High effort: allocates more reasoning tokens in it's budget to enable reasoning at a more complete and thorough level.

For each combination of problem and reasoning control, we let the LLM generate 20 Kokkos codes (samples) to analyze the effect of multiple sampling.

3.3 Evaluation Metrics

We use the total amount of tokens used for code generation to assess the cost of an inference-time reasoning control. They include tokens inputted to and outputted from the LLM. This also means that we do not include human manual review cost. Thus, it is important to keep in mind to interpret the costs we mention as token efficiency under benchmark-style automated validation, rather than end to end developer effort. Unfortunately, we have no data on the detailed breakdown, so we show only the total amount of input and output tokens.

There are two general ways of measuring the correctness of codes generated by LLMs: comparing the similarity of the generated code to a reference code [13] and evaluating the functional correctness of the generated code [2]. In this paper, we use the latter approach by using the metric pass@k. Pass@k is the probability that at least one sample passes a given unit test when generating k independent code samples. Mathematically, pass@k can be described as follows.

$$\text{pass@}k = \frac{1}{|P|} \sum_{p \in P} \left[1 - \frac{\binom{N-c_p}{k}}{\binom{N}{k}} \right]$$

where $|P|$ is the set of prompts, N is the number of samples generated per prompt, c_p is the number of correct samples for prompt p, and k is the number of samples to consider [9].

Like Tanaka's work [16], we provide a detailed analysis of generated codes by categorizing them into the following classes.

Pass: the code generated passed all test cases.
Logical error: the code generated completed execution normally (i.e., exit code 0), but failed at least one test case.
Time Limit Exceeded: the code runs longer than the given time limit (5 minutes).
Runtime error: the code cannot exit normally.
Compilation error: the code cannot be compiled to an executable binary.

To evaluate the performance of the generated code, we use the minimum, maximum, and average speedups. For each tested parallel configuration, we take the maximum (or average or minimum) observed runtime across repeated runs and then compute the speedup from that value, yielding a worst-case (or average or best-case) estimate. We execute each pass code on the CPU with 1-32 threads. We note that each thread is executed on a different physical core for 1 to 16-thread execution, while two threads share one physical core during the 32-thread execution.

Table 2. ParEval benchmark suite

Problem Type	Description	Problems
Sort	Sort an array or subarray of values; in-place and out-of-place.	complex number sort, finding k-th element, sorted vector's index, sort by key, sort but ignore zero
Scan	Scan operations, such as prefix sum, over an array of values.	prefix sum, scan min and replace, sum of prefix sum, reverse prefix sum, largest contiguous subarray sum
Dense Linear Algebra	Dense linear algebra functions from all three levels of BLAS.	LU decomposition, linear system solver, GEMM, AXPY, GEMV
Sparse Linear Algebra	Sparse linear algebra functions from all three levels of BLAS.	linear system solver, SpMM, SpMV, AXPY, ludecomp
Search	Search for an element or property in an array of values.	search last struct by key, search value, find the closest number to pi, search first even number, search xor
Reduce	Reduction operation over an array dimension, such as computing a sum.	XOR, the product of the vector x with every odd indexed element inverted, mean, smallest odd number, the sum of the minimum value at each index of vectors x and y
Histogram	Binning values based on a property of the data.	pixel counting, histogram 0-100, counting the number of cartesian points in each quadrant, counting first letter, quartile counting
Stencil	One iteration of 1D and 2D stencil problems, such as Jacobi relaxation.	xor, grayscale image convolution with the edge kernel,3 point 1D Jacobi stencil, 3-point 2D Jacobi stencil, game of life
Graph	Graph algorithms, such as component counting.	edge counting, counting the number of vertices in the largest component, highest node degree, counting connected components, shortening path
Geometry	Compute geometric properties, such as convex hull.	convex hull, perimeter of convex hull, smallest triangle, 2D closest pair, 1D closest pair
Fourier Transform	Compute standard and inverse Fourier transforms.	inverse FFT, DFT, conjugate, split FFT, FFT
Transform	Map a constant function to each element of an array.	ReLU, odd value division by 2, x to 1/x, squared value, map

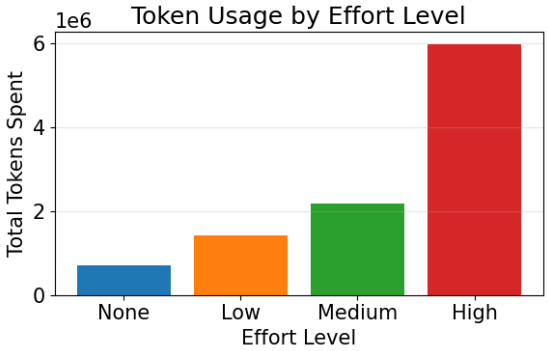


Fig. 1. Amount of the total tokens used to generate Kokkos codes for different inference-time reasoning levels.

Additionally, to understand the token-efficiency trade-off between reasoning effort and multiple sampling, we compare no-effort multi-sampling to single sampling with a higher effort level. Specifically, we compute pass@k for the no-effort level (where k ranges from 1 to 20) and evaluate whether generating k cheap samples can match or exceed the performance of a single expensive sample. For each configuration, we calculate a *value ratio* defined as follows.

$$\text{Value Ratio} = \frac{\text{pass@k}_{\text{no-effort}} / \text{pass@1}_{\text{higher-effort}}}{(k \times \text{cost}_{\text{no-effort}}) / \text{cost}_{\text{higher-effort}}}$$

A value ratio ≥ 1.0 indicates that no-effort multi-sampling achieves at least equivalent pass-per-token efficiency compared to a given higher effort level. A higher value ratio means multiple sampling is better than an increase in inference-time reasoning levels.

4 Experimental Results

4.1 Cost of Inference-Time Reasoning

Figure 1 shows the relative cost (i.e., the number of tokens) of generating Kokkos codes for different inference-time reasoning levels. It can be observed that the high effort level shows a substantial increase in the number of tokens compared to the other effort levels. More specifically, the high effort level needs 1.74x and 7.51x greater number of tokens than the medium and no effort levels, respectively. This means that the use of the high effort level for Kokkos code generation is almost equivalent to generating 7 or 8 samples without reasoning effort in terms of cost. Therefore, the use of a high effort level should be avoided if it has a small or no impact on the correctness and performance of generated codes.

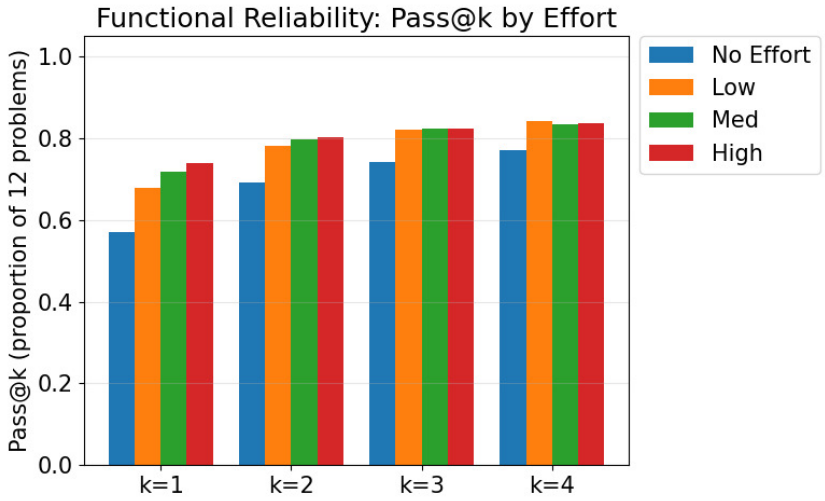


Fig. 2. Impact of inference-time reasoning levels and number of samples on pass@k.

4.2 Impact on Correctness

Figure 2 shows the impact of inference-time reasoning and multiple sampling on the correctness of generated codes. The figure shows that generating Kokkos code with GPT-5.1 is still rather unreliable, especially in the case of using only one sample, given that even at the high effort level, it still has a probability of 74.1% to generate functionally correct code. We can observe an upward trend in correctness against reasoning effort levels, especially at k = 1, which supports the idea that higher inference-time reasoning increases the probability of an LLM producing functionally correct code, with a 9.3% increase from low to high. Multiple sampling also increases the probability of generating functionally correct code; for example, sampling code 4 times at the low effort level increases the probability by 24%. The same trend, in which more sampling gives a better probability increase as opposed to using the high effort level, can also be seen at the no effort and medium effort levels.

Given the observed trend that multiple samples can outperform higher inference-time reasoning, we still need a more objective comparison between the two methods. For that, we have provided the cost efficiency graph as shown in figure 3. While no-effort multi-sampling achieves the same cost-efficiency as medium effort at k=3, matching high-effort reasoning requires k=10 samples. This suggests that lower-reasoning effort generally uses tokens more efficiently for producing functionally correct code. The value ratio declines monotonically with k due to the diminishing returns of pass@k against linearly growing cost. It means that the first sample is always the most token-efficient. In practical terms, the token budget of a single high-effort query (roughly 850% of one NE sample) could fund approximately 8 no-effort samples, which show slightly higher correctness than

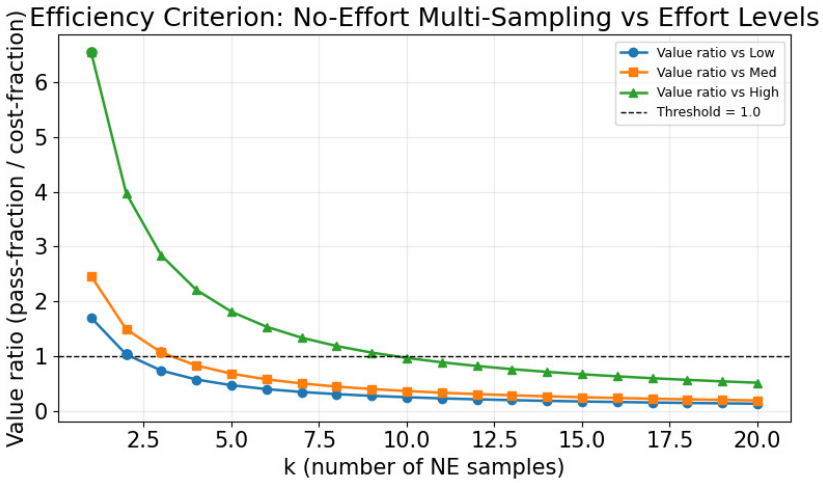


Fig. 3. Impact of inference-time reasoning levels and number of samples on value ratio.

one no-effort sample. This efficiency gap likely arises because the additional reasoning tokens at higher effort levels do not proportionally reduce error rates for these parallel programming tasks. We note that this analysis is limited to functional correctness; thus, higher reasoning effort may still yield advantages in the other aspects of code quality, such as performance and maintainability. We also note that in practical software development, on the worst case scenario, developers need inspect and validate the correctness of each of the sample produced by the LLM. This introduces a layer of human cost which may penalized the multiple sampling method heavily. Therefore, our results is suitable for environment in which strong automatic validation is available.

Figure 5 shows the breakdown of generated codes for different effort levels. The figure shows that most generated codes pass all the test cases and many errors happen at compile time. One interesting finding from our analysis is that for the top 4 problems that caused the most errors in our experiment (inverse Fourier transform, count quartile, sparse AXPY, search XOR), every attempt resulted in an error (i.e., 100% failure rate). This suggests that some problems can cause systematic and consistent failures regardless of the approach. Upon further investigation of this phenomenon, we find that the four problems which has 100% failure rate contains bugs in the benchmark itself. Here are the bugs found on the test cases:

Inverse Fourier Transform: The prompt used for this problem assumes that there exist a function called fft but said function isn't included in the driver, thus all of the generated code which uses the fft function has problem resolving what this fft function is.

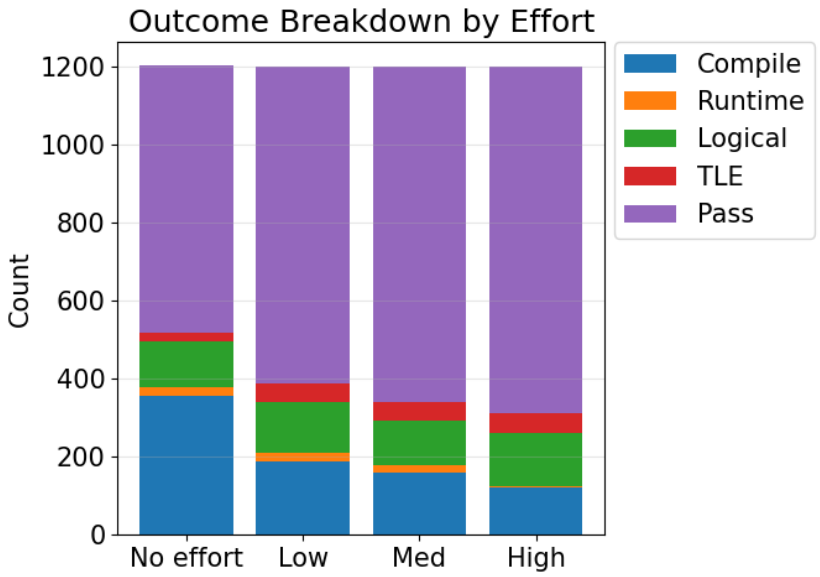


Fig. 4. Breakdown of generated codes.

Count Quartile: The driver only allocated an array of four elements to accommodate the result of the generated code and the baseline code, but the driver checks for elements larger than four.

Sparse AXPY: The prompt provided in the benchmark forces a non-const reference parameter using the & symbol, even though how the driver sends `Kokkos::View` is not compatible with that form. More specifically, when `Kokkos::View<Element*>` is sent to `Kokkos::View<Element*>&`, C++ needs to make a temporary conversion to const-view even though this temporary value cannot be assigned to the non-const reference.

Search XOR: The driver does not initialize `ctx->x`, but instead, it incorrectly initializes `ctx->x_host`.

Upon investigating the comparison of functional correctness of the generated codes on each problem class, we found that FFT (Fast Fourier Transform) problems have an abnormally low pass@4 score compared to all of the other problem classes. This pass@4 score is then followed by the histogram, search, and sparse linear algebra class which has about the same pass@4 score. It's notable that the four classes mentioned previously contain each of the four problems with a 100% failure rate; this is one of the reasons why the four have low scores. That said, FFT problems still have significantly lower scores compared to the other three. Previous evaluation by [9] using GPT-4.0 also shows that Kokkos struggles with Fast Fourier Transform problems.

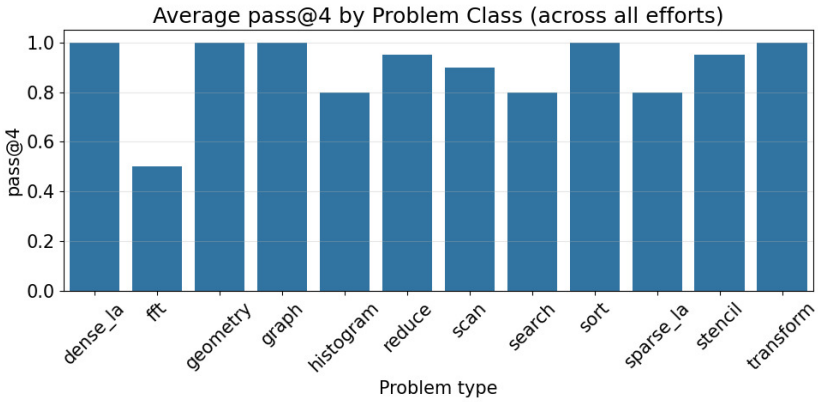


Fig. 5. Average pass@4 of each problem class.

4.3 Impact on Performance

Figure 6 shows the minimum, maximum, and average speedup of generated codes over thread count. The figure shows that as the thread count increases, strong-scaling speedup exhibits diminishing returns because of Amdahl's law [1, 6]. The figure also shows that even in the worst-case scenario, the generated codes exploit some parallelism, as we can see an upward trend in minimum speedup against an increase in thread count.

We found three anomalies in the figure. First, the speedup tends to drop at the 32-thread mark. This is clearly affected by simultaneous multithreading, which can increase contention on shared resources within a core, thus reducing effective throughput and speedup [14]. Second, the low effort level tends to produce the lowest speedup compared to the other effort levels, including the no effort level. This suggests that a low effort level is unsuitable for parallelization tasks. The reason for this under-performance is that parallelization efficiency, unlike functional correctness, is largely affected by reduced thinking effort. The low effort level produces code that has correctness similar to higher effort levels, yet it fails to scale with thread count——suggesting that generating correct and efficient parallel code is fundamentally different for reasoning models. This is also supported by the third anomaly, where the no effort reasoning level outperforms the medium or even high effort levels in most cases.

5 Conclusion

In this paper, we showed the impact of inference-time reasoning on LLM-generated Kokkos code while comparing it to the multiple sampling method. Our experimental results showed that increased inference-time reasoning improved pass@1, but the increase in token cost at the high effort level was substantial (1.74× and

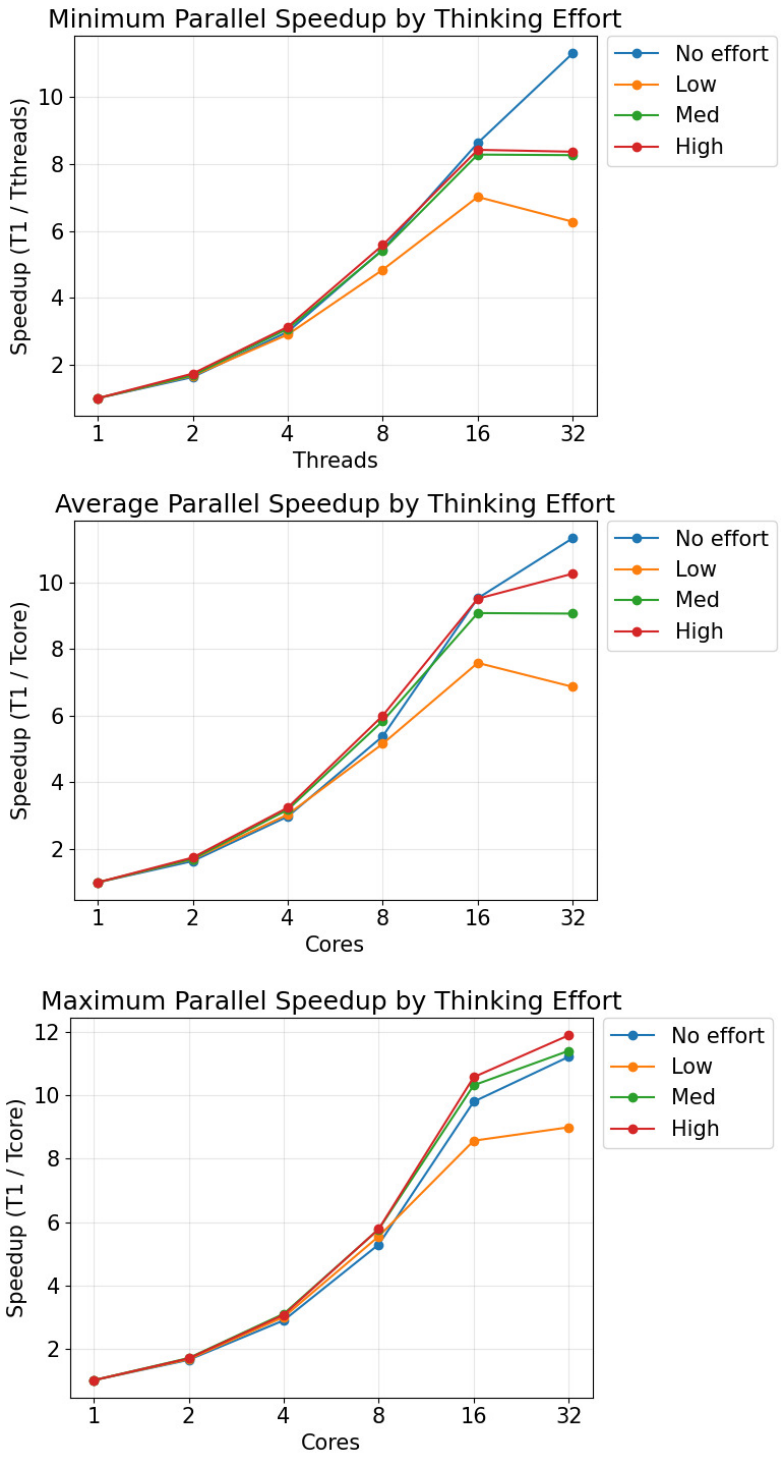


Fig. 6. Minimum, maximum, and average speedup of generated Kokkos codes across inference-time reasoning levels.

7.51× compared to the medium and no effort levels, respectively); thus, the advantage of using higher inference-time reasoning for accuracy is not proportional to the cost. As for cost efficiency, under automated validation, using multiple sampling at the no effort level was more beneficial than using a higher effort level.

Analysis of the taxonomy of errors showed that the main failures happened at compile time. Aside from that, we also observed some problems that were able to trigger a systematic failure (100% failure rate) regardless of effort levels. Upon further investigation, this is because of bugs found in the ParEval benchmark. In terms of performance, the generated codes that passed test cases showed mostly good scalability against thread count, but they still experienced a marginal decrease in speedup due to the limit of strong scaling. We also observed a few counterintuitive aspects in the relationship between performance and reasoning effort levels. This finding supports our assumption that generating *correct* and *efficient* parallel code is fundamentally different: an increase in reasoning effort is consistently helpful for improving the correctness of the generated code, but it does not guarantee improvement in parallelization efficiency or scalability.

Practically, for LLM-assisted Kokkos code development with automated validation, the most token-efficient strategy is to prioritize multiple sampling with a low effort (or no effort) level to generate correct code, and then select the best code through validation and performance testing. On the other hand, a high effort level should be used conservatively and only in limited cases, such as when multiple sampling is costly or when non-functional qualities, such as code readability and structure, are a priority.

Our future work will extend this research to other parallel codes and other LLM test-time search methods, namely the iterative refinement methods like the ones that can be found on sequential refinement or self-refine[15, 7]. Furthermore, we will investigate the impact of inference-time reasoning on parallel code generation for other LLMs.

Acknowledgments. This work was partially supported by JST, PRESTO Grant Number JPMJPR22P9. The author gratefully acknowledges the support provided by the Japan Student Services Organization (JASSO) scholarship through the UEC JUSST program.

References

1. Amdahl, G.M.: Validity of the single processor approach to achieving large scale computing capabilities. In: Proceedings of the April 18–20, 1967, Spring Joint Computer Conference. pp. 483–485. AFIPS '67 (Spring), ACM (1967). https://doi.org/10.1145/1465482.1465560
2. Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X., Lou, Y.: Evaluating large language models in class-level code generation. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24 (2024)

3. Ellingwood, N.D., Siefert, C., Jermann, E.J.: Evaluation of llm-generated kokkos code using compile-time and run-time testing. Tech. rep., Sandia National Laboratories (SNL-NM), Albuquerque, NM (United States) (2025)
4. Godoy, W.F., Valero-Lara, P., Teranishi, K., Balaprakash, P., Vetter, J.S.: Evaluation of openai codex for hpc parallel programming models kernel generation. In: Proceedings of the 52nd International Conference on Parallel Processing Workshops. pp. 136–144 (2023). https://doi.org/10.1145/3605573.3605600
5. Godoy, W.F., Valero-Lara, P., Teranishi, K., Balaprakash, P., Vetter, J.S.: Large language model evaluation for high-performance computing software development. Concurrency and Computation: Practice and Experience (2024). https://doi.org/10.1002/cpe.8269
6. Hill, M.D., Marty, M.R.: Amdahl's law in the multicore era. Computer **41**(7), 33–38 (2008). https://doi.org/10.1109/MC.2008.209
7. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., et al.: Self-refine: Iterative refinement with self-feedback. Advances in neural information processing systems **36**, 46534–46594 (2023)
8. Mišić, M., Dodović, M.: An assessment of large language models for openmp-based code parallelization: A user perspective. Journal of Big Data **11**(161) (2024). https://doi.org/10.1186/s40537-024-01019-z
9. Nichols, D., Davis, J.H., Xie, Z., Rajaram, A., Bhatele, A.: Can large language models write parallel code? In: Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing. pp. 281–294 (2024)
10. Nichols, D., Marathe, A., Menon, H., Gamblin, T., Bhatele, A.: Hpc-coder: Modeling parallel programs using large language models. In: ISC High Performance 2024 Research Paper Proceedings (39th International Conference). pp. 1–12. Prometeus GmbH (2024)
11. OpenAI: Reasoning models. https://developers.openai.com/api/docs/guides/reasoning/ (2025), accessed: 2026-02-28
12. Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M.: The impact of ai on developer productivity: Evidence from github copilot. arXiv preprint arXiv:2302.06590 (2023). https://doi.org/10.48550/arXiv.2302.06590
13. Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S.: Codebleu: a method for automatic evaluation of code synthesis. arXiv preprint arXiv:2009.10297 (2020)
14. Saini, S., Jin, H., Hood, R., Barker, D., Mehrotra, P., Biswas, R.: The impact of hyper-threading on processor resource utilization in production applications. In: 2011 18th International Conference on High Performance Computing. pp. 1–10. IEEE (2011)
15. Singh, G., Guha, A., Kailkhura, B., Menon, H.: Can test-time compute help llms write low-resource parallel code better? In: NeurIPS 2025 Fourth Workshop on Deep Learning for Code
16. Tanaka, R., Yamaki, H., Miwa, S., Honda, H.: Analysis of mpi parallel code generated by gpt-4o. In: International Conference on High Performance Computing. pp. 639–652. Springer (2025)
17. Valero Lara, P., Young, A., Vetter, J.S., Jin, Z., Pophale, S., Alaul Haque Monil, M., Teranishi, K., Godoy, W.F.: Chathpc: Building the foundations for a productive and trustworthy ai-assisted hpc ecosystem. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. pp. 458–474 (2025)

18. Zaremba, W., Nitishinskaya, E., Barak, B., Lin, S., Toyer, S., Yu, Y., Dias, R., Wallace, E., Xiao, K., Heidecke, J., et al.: Trading inference-time compute for adversarial robustness. arXiv preprint arXiv:2501.18841 (2025)